AgentProf: Semantic Profiling for AI Agents

Anonymous submission

Abstract

AI agents orchestrate long-running activities across users, tools, and system resources, producing growing populations of execution histories. Developers need population-level answers about where failures concentrate, which workflows trigger unsafe effects, and which task categories consume the most budget. Unlike stable code paths, natural-language intent lacks shared identity, and native execution nesting does not provide a reusable cross-run hierarchy of semantic responsibility. Existing agent tools trace runs, group metadata, and aggregate metrics, but do not recursively construct shared semantic responsibility while retaining LLM/tool evidence and conserved resource measures in a standard profile. Agent observability needs profiling, not only debugging. AgentProf realizes this insight through a *semantic operation stack model*: backend-neutral recursive annotations identify shared responsibilities over an ordered source tree, operation stacks compose those responsibilities with native evidence, and additive resource views emit pprof-compatible profiles. We evaluate AgentProf on 405 human-staged CodeTraceBench trajectories, three complete public localization benchmarks, 287 OSWorld-Human sessions, two population case studies, and four public cost workloads. Automatic Agent annotation raises ordinary B^3 F1 against human stages from 0.541 for raw action and 0.663 for recurrence to 0.704. On three complete localization workloads used for protocol development, AgentProf raises MAP over benchmark-native direct diagnostics by 0.031, 0.107, and 0.117, but is statistically indistinguishable from an information-matched raw-action plus source-evidence refinement. Evaluated task-family and action backends reach 0.695 and 0.498 macro-F1, label-free recurrence reaches 0.680 boundary F1 and 0.786 B^3 F1 on OSWorld-Human, and, after marks are fixed, AgentProf constructs a 27,765-operation profile in 1.16 s.

Introduction

AI agents (Yang et al. 2024; OpenAI 2025; Anthropic 2025; Xie et al. 2024) orchestrate multi-step activities across a high-level intent layer (prompts, LLM calls, and tool invocations) and a low-level system-effects layer (process spawns and file and network I/O). As agents evolve from short scripts into complex systems, production teams accumulate many trajectories.

Developers lack population-level answers about agent quality, safety, and cost. Which categories of tasks consume the most token budget? Where do failures concentrate across workflows? Which behavioral patterns trigger unsafe system effects? At scale, manual trajectory inspection is slow and expensive (Lù et al. 2025), while LLM judging requires a separate evaluator pass per trajectory (Lù et al. 2025; Zheng et al. 2023).

Agent trajectories lack two structures needed for cross-run profiling. Responsibility lies in semantic categories such as task intent and tool operation rather than stable function names. Natural-language prompts provide no common identifier for shared intent. Native execution nesting may exist, but it does not provide a reusable cross-run hierarchy of semantic responsibility. A prompt sequence may represent multiple tasks, while one task may belong to a larger task. Thus adapting profiling requires stable semantic identifiers and an attribution hierarchy that does not depend on a runtime stack.

Existing agent observability lacks this profiling abstraction. Tools provide rich tracing, metadata aggregation, and population dashboards (LangChain 2024; Langfuse 2024; OpenTelemetry 2024; Datadog 2025), and some derive hierarchical categories across traces and aggregate latency, cost, or evaluation metrics (LangChain 2026; Datadog 2026), while recent research canonicalizes actions into process profiles or cross-run graphs for monitoring and recovery (Shu et al. 2026; Liu et al. 2026a; Zhong, Saxena, and Raghunathan 2026; Nian et al. 2026; Gao et al. 2026; Wang et al. 2026b). These systems establish semantic grouping, metric rollups, and cross-run analysis. They do not combine recursively annotated semantic responsibility, source-linked LLM/tool evidence, conserved additive measures, and standard profiler output over the same histories.

Agent observability needs profiling, not only debugging. Profiling applies to agents when a profiler projects resource measures onto stable semantic tags in an attribution hierarchy. Our *semantic operation stack model* provides this structure through source nodes, recursive operation annotations, and operation stacks. Source nodes retain native session, prompt, LLM-call, and tool-call relations and additive measures. An interchangeable backend names the session scope,

each user-prompt scope, and recursively refined intervals as operations. An *operation stack* emits the agent, those semantic operations, and the covered LLM/tool evidence; raw session and prompt IDs remain labels rather than visible frames that fragment aggregation (Figure 2).

We implement this model in AgentProf, an offline profiler that compiles local agent trajectories into pprof-compatible profiles (Google 2024) (Figure 1). *Operation annotation* lets an Agent, a language model, or a non-LLM backend mark nested source intervals with shared semantic names. *Stack construction* deterministically emits `agent`→session-level operation→prompt-level operation→recursive operations→LLM call→tool call. Raw session and prompt identities remain pprof labels for drill-down.

Across all 405 released CodeTraceBench trajectories reconstructable from source (Li et al. 2026), automatic Agent annotation reaches 0.704 ordinary B^3 F1 against human stages, compared with 0.663 for multi-resolution recurrence and 0.541 for raw action. Across three complete public workloads used for protocol development, AgentProf refines benchmark-native direct diagnostic ties and raises MAP by 0.031, 0.107, and 0.117, respectively (Fan et al. 2026; Wang et al. 2026a; Chen et al. 2026a). An information-matched raw-action plus source-evidence refinement is statistically tied on all three workloads. On 287 OSWorld-Human development sessions (Abhyankar, Qi, and Zhang 2025), label-free recurrence reaches 0.680 boundary F1 and 0.786 B^3 F1. On the AgentBoard and ASE populations, task-family and action backends reach 0.695 and 0.498 macro-F1, respectively (Ma et al. 2024; Bouzenia and Pradel 2025). On four public workloads, after marks are fixed, AgentProf constructs a 27,765-operation profile in 1.16 s.

This paper makes three contributions:

1. **Semantic operation stack model.** Recursive semantic responsibility over source-native evidence supplies agent observability's missing profiling layer (Background and Motivation; Design).
2. **System: AgentProf.** A backend-neutral annotation workspace composes recursively marked semantic intervals with source-native hierarchy and produces pprof-compatible profiles (Implementation).
3. **Evaluation.** Automatic operation structure improves human-stage agreement on CodeTraceBench, and operation profiles supplement benchmark-native direct diagnostics on all three localization workloads while tying an information-matched raw-evidence refinement. A repeated-task resource case and a population-scale differential case expose high-cost unfinished work and success–failure differences. After marks are fixed, AgentProf profiles 27,765 operations in 1.16 seconds (Evaluation).

Background and Motivation

LLMs and AI Agents

A typical agent trajectory interleaves two layers of activity (Yang et al. 2024; OpenAI 2025; Anthropic 2025; Zheng et al. 2025). At the intent layer, an agent receives prompts, issues LLM calls, and invokes tools for code execution, search, and file editing. Tool invocations trigger lower-layer process, file, and network effects (Zheng et al. 2025). Trajectories repeat this cycle, and teams accumulate many trajectories.

System Profiling

Unlike single-execution debugging, profiling aggregates measurements to attribute resources to responsible entities. Traditional profilers sample execution, attach each sample to its call stack, and fold identical stacks (Google 2024; Gregg 2011). Flame graphs (Gregg 2011) and pprof (Google 2024) call graphs visualize aggregate cost by width or node size, while Pivot Tracing (Mace, Roelke, and Fonseca 2015) dynamically selects and groups measurements across causally related events. These tools start from stable code identifiers and runtime stacks, although pprof can also promote `tagroot`/`tagleaf` values to pseudo-frames (Google 2024). Agent profiles must instead derive stable semantic fields and connect them to downstream effects.

Challenges for Agent Profiling

Existing tools (LangChain 2024; Langfuse 2024; OpenTelemetry 2024) record agent events as per-execution span trees for single-run debugging. AgentSight (Zheng et al. 2025) bridges intent and system-effect layers by correlating extended Berkeley Packet Filter (eBPF) system events with intercepted LLM traffic, but the resulting recordings still lack aggregate profiling.

Agent profiling faces two challenges. First, responsibility lies in semantic categories such as task intent and workflow phase, not code paths. Standards represent application-supplied span attributes (OpenTelemetry 2024; Arize AI 2024), but natural-language histories lack stable categories because different prompts or tools can express the same intent.

Second, agent trajectories lack a runtime call hierarchy for semantic attribution. In CPU profiling, the stack attributes `malloc`'s cost simultaneously to `parse`, `process`, and `main` (Google 2024; Gregg 2011). A prompt sequence may contain one or several tasks within a larger workflow, yet no runtime mechanism marks its boundaries or attribution granularity. RQ1 evaluates how this missing hierarchy affects attribution, while RQ3 evaluates constructed groups and boundaries.

Design Requirements for Agent Profiling

Traditional profiling relies on three mechanisms that the call stack supplies automatically. Agent profiling requires all three without a call stack:

D1: Conserved multi-resource attribution. Each atomic source record must retain additive resource measures, such as tokens, time, file effects, and network effects, so regrouping never changes their total mass.

D2: Shared semantic identity. Responsibilities that recur in different sessions need the same short semantic name so that their costs fold together rather than remaining session-local.

D3: Hierarchy without evidence loss. The profiler must derive variable-depth semantic responsibility from the

native trace while retaining covered LLM/tool evidence as leaf frames and raw session/prompt identities as drill-down labels.

Design

The design has three explicit objects: an ordered source tree, nested semantic annotations, and a weighted pprof stack. A source adapter constructs the first without discarding content or native parent relations. Any annotation backend constructs the second without rewriting source nodes. The deterministic profiler validates both, emits agent and semantic-operation frames followed by covered LLM/tool evidence, and folds equal paths into the third.

This separation addresses D1–D3 directly. Additive measures remain attached to source nodes; shared semantic names create cross-session identity; LLM/tool frames preserve visible evidence; and unique session, prompt, call, and event IDs remain pprof labels. Switching from operation count to tokens, time, file effects, or network effects replays the same annotation and changes only stack width.

Semantic Operation Stack Model

Let a source node be $n = (id, parent, kind, data, m)$ in an ordered forest. The *kind* is a session, prompt, LLM call, tool call, effect, or an adapter-defined atomic event. The *data* retains the source content needed to interpret the event, and m maps resource names to nonnegative additive measures. Source-parent links encode observed execution structure; they never encode an inferred semantic operation.

Recursive annotation covers every source node with exactly one active semantic leaf and its enclosing operation ancestors; let this path be $A(n)$. Let $g(n)$ be the agent identity inherited from the source root, and let $E(n)$ be the retained evidence ancestry after removing raw session and prompt containers—normally the covered LLM call, tool call, and effect. The emitted operation stack is $\sigma(n) = \langle \textit{agent} : g(n) \rangle \parallel A(n) \parallel E(n)$, where $\parallel$ denotes concatenation. Raw session and prompt IDs are attached as labels $\lambda(n)$ rather than visible frames. Equal semantic prefixes can therefore aggregate across sessions without losing the evidence needed to locate a contributing call or tool event.

We formalize a resource profile as (φ, σ, w_r): predicate $\varphi(n)$ selects source nodes, $\sigma(n)$ supplies the composed stack, and $w_r(n) = m_n[r] \geq 0$ selects one additive resource. Folding merges equal stacks and sums w_r. The default operation-count view assigns one unit to each adapter-defined atomic operation; token, time, file-effect, and network-effect views select their corresponding measures. Because A and E remain fixed, changing r changes only widths, not boundaries or semantic names.

Recursive Operation Annotation

The source adapter preserves an ordered tree of sessions, prompts, LLM calls, and tool calls. Semantic operations are nested intervals over this tree rather than replacements for its nodes. An annotation at source node s contains a semantic name, the start node of its enclosing annotation, and the first source node after its interval. The exclusive end may be omitted when the interval reaches its parent's end. Every source root must begin a session-level operation, and every prompt must begin a prompt-level operation. Further nested intervals define the variable-depth suffix of the operation path. Equal path prefixes fold across sessions; covered LLM/tool nodes remain below the path, while raw session and prompt IDs remain labels for evidence drilldown.

Annotation proceeds coarse to fine. A backend first names the complete session and each user-prompt scope, then selects one interval whose current name hides a useful responsibility change, marks source-visible boundaries, names the resulting child intervals, and repeats only where another distinction helps. Branches stop independently, so depth is neither fixed nor optimized directly. A later iteration may rename, merge, or remove an earlier split. The default Agent-assisted backend reads source content and existing paths to make these decisions. A plain language model, deterministic rules, change-point methods, and the label-free recurrence constructor use the same annotation contract. The profiler therefore does not depend on one model or inference service.

The evaluated Agent backend instantiates this contract with a fixed binary recursive policy. For an interval, it receives target-blind task text, the active ancestor path, and every source-visible turn summary in that interval, then returns either STOP or SPLIT(*non-first turn*, *left name*, *right name*). A child name equal to the current frame stays, one equal to an ancestor pops to that frame, and a new name pushes one frame; equal resolved children fail closed. Nonempty children recurse independently. A one-turn interval, STOP, or a split whose two sides both resolve to the unchanged path terminates the branch. There is no depth cap, leaf cap, length threshold, or score-tuned contraction. After recursion, the materializer emits a sparse mark only when the complete path changes, thereby merging adjacent identical paths but never nonadjacent or unequal ones.

The complete CodeTraceBench A2 run uses independent Codex Agent workers under one fixed source-only annotation instruction, followed by root merge and validation. Official stages, outcomes, recurrence assignments, and scores remain hidden until all 405 trajectories are materialized. A2 adds one deterministic source-only representation repair: when a one-turn root-only mark is immediately followed by its first responsibility, the first turn receives the complete root–responsibility path. Before folding, one fixed source-only action–object map canonicalizes the A2 display identity. A boundary-safe refinement retains a canonical action and the minimum source words needed whenever two adjacent paths would otherwise become equal; unresolved collisions fail closed. Equal canonical names receive one stable operation ID across sessions. The complete replay reduces 5,537 open-vocabulary names to 1,434 two- or three-word identities with zero adjacent path collision, while preserving every temporal mark occurrence.

Implementation

AGENTPROF is an offline Rust command-line interface that implements the semantic operation stack model (Figure 1).

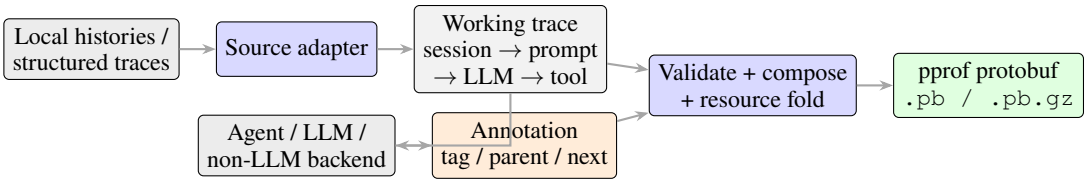

Figure 1: AgentProf pipeline. A source adapter preserves the native trace tree. An Agent, LLM, or non-LLM backend writes the same small annotation contract. The CLI validates and composes semantic paths with source evidence, folds one additive resource measure, and emits only a standard pprof profile.

Input reconstruction. Source-specific adapters reconstruct an ordered tree from local Codex and Claude Code histories, AgentSight (Zheng et al. 2025) recordings, or public trajectory datasets. Each adapter preserves replay-stable node IDs, source content or a readable reference to it, native parent links, input order, and available additive measures. Structured operation JSONL is accepted by materializing its records as adapter-defined atomic source nodes rather than inventing missing prompt or call relations.

Annotation workspace. The backend reads `trace.jsonl` and writes only `annotation.json`. The former retains source content, source-parent links, additive measures, and the current derived path; the latter maps source start IDs to `tag`, semantic `parent`, and exclusive `next`. AgentProf validates that references exist and intervals are nested, noncrossing, and collectively cover every source node, then atomically updates derived paths and the current folded aggregate. It also emits nonblocking hierarchy warnings for a recursive operation with only one explicit semantic child, a large flat fan-out with little recursive refinement, or an optional semantic leaf that spans at least eight tool calls. These diagnostics expose redundant, prematurely flat, or coarse annotations without forcing artificial depth or blocking profile generation. The same annotation can be replayed with operation count, tokens, time, file effects, or network effects without changing a boundary or semantic name.

Non-LLM recurrence backend. The Rust inducer counts adjacent action transitions in reference sessions and scores each transition by normalized pointwise mutual information (NPMI), $\mathrm{NPMI}(a,b) = \ln[p(a,b)/(p_L(a)p_R(b))]/ - \ln p(a,b)$, with all three probabilities defined over the same transition sample space (Bouma 2009).

Occurrence-weighted one-dimensional k-means with $k = 2$ (MacQueen 1967) initializes at the minimum and maximum scores, assigns distance ties to the lower center, and uses the converged midpoint as a global cutoff. The inducer applies the same procedure to action-changing transitions for a second cutoff. Same-action pairs use the global cutoff, whereas different-action pairs use the smaller cutoff, which can remove a boundary produced by the global rule but cannot add one. When every reference and target operation also has a nonempty `action_detail`, the inducer fits the same recurrence model to the compound signature. Detailed continuity may remove a coarse boundary but can never add one. Missing, unseen, or weak detail falls back exactly to the coarse decision. Segment names remain coarse run-length-compressed action sequences. The inducer reads only session identity, input order, action, and optional action detail, not target annotations or resource weights.

An optional reference-calibrated recurrence mode uses the same NPMI score but selects one scalar cutoff that maximizes per-operation B^3 F1 on disjoint reference groups. The mode enumerates score intervals, resolves exact ties toward the smallest cutoff, and never reads target groups. Label-free k-means remains the default.

Profile export. AgentProf emits one pprof protobuf profile for existing tools such as `go tool pprof`; it does not add a visualization frontend or a second export format.

Evaluation

Research questions. The evaluation addresses four research questions: **RQ1:** Does semantic profiling improve resource attribution? **RQ2:** Does profiler output correspond to real problems? **RQ3:** How accurately do automatic backends recover operation structure? **RQ4:** What is the cost of constructing a semantic profile? Each RQ subsection pairs its protocol with the strongest answer its evidence supports.

We evaluate three data classes separately. **Real inputs (RQ1 and case studies)** include 41 long-horizon coding-agent sessions and 440 real mixed-outcome web-agent trajectories. **Annotated trajectories (RQ3 and RQ4)** include 15 real-agent or human web, API, coding, and mobile/GUI execution families, plus 405 CodeTraceBench trajectories with 20,866 operations (Lù, Kasner, and Reddy 2024; Deng et al. 2023; Xu et al. 2025; Yao et al. 2022; Li et al. 2023; Qin et al. 2024; Yao et al. 2025; Yang et al. 2024; Li et al. 2024; Lu et al. 2025; Liu et al. 2026b; Lù et al. 2025; Chen et al. 2026b; Wang et al. 2025; Abhyankar, Qi, and Zhang 2025). **Problem-localization benchmarks (RQ2)** provide independent targets over 27,346 operations (Fan et al. 2026; Wang et al. 2026a; Chen et al. 2026a). The RQ4 union contains 27,765 operations (Lù et al. 2025; Chen et al. 2026b; Abhyankar, Qi, and Zhang 2025; Wang et al. 2025).

RQ1: Multi-Resource Attribution

RQ1 asks whether semantic profiling improves resource attribution. We test one necessary consequence: whether a fixed semantic hierarchy reveals materially different bottlenecks when the additive resource measure changes. The source population contains 41 real long-horizon agent trajectories, 3,146 source-native turns, and 5,750 operations.

Within it, three independent executions repeat the same Git-deployment task. The automatic Agent backend assigns 96 recursive annotations to their 735 source nodes. The resulting hierarchy reaches semantic depth five without a fixed limit. The CLI reports no unary or flat-fan-out warning and 27 advisory coarse-leaf warnings. This establishes the necessary multi-resource attribution capability on a repeated real task; it does not claim superiority over every attribution interface or generalize the measured ratios beyond this population.

Case Study 1: Long-Horizon Agent Diagnosis. Figure 2 uses a stock-pprof focus query to retain all three executions of one repeated Git-deployment task rather than selecting one trace.

The focused task contains 489 operations and 4,558,192 provider-reported tokens from OpenHands/Claude, OpenHands/DeepSeek, and Terminus2/DeepSeek. By count, Terminus2 contributes 56.24% and OpenHands 43.76%; by tokens, OpenHands contributes 86.62%. The repeated `diagnose authentication` operation directly contains 97 operations and 1,936,828 tokens; its complete recursive subtree contains 105 operations and 2,103,587 tokens (21.47% and 46.15% of the focused task). Source drilldown shows both OpenHands executions returning to this responsibility after control or fallback attempts; all three executions verify web deployment or substitute transport paths, but none establishes the requested password-authenticated `git@localhost` path. As a post-hoc organization control, we replay the same 489 source operations and both weights through the current binary with common `project, agent` prefixes, `call, tool` evidence leaves, and three alternative middle organizations: native `session, prompt`, coarse `action kind, raw action`, or the accepted semantic operation path. All six profiles conserve exact mass and open in stock pprof. The fixed SSH-responsibility members occupy 105 distinct source calls in the native hierarchy and span six coarse action kinds; the largest coarse branch contains only 39.42% of their tokens. At the next coarse level, 102 of 105 members are merely `run`, together with 97 unrelated operations. Only the semantic organization supplies one focusable cross-run responsibility while retaining those same source labels and leaves. Because the responsibility was selected from the prior semantic case, this matched projection establishes an organization difference, not independent discovery accuracy. Thus, the fixed hierarchy exposes a bottleneck that an operation-count view substantially understates: the SSH-diagnosis subtree is roughly one fifth of work by count but nearly half by tokens. Source drilldown explains the disparity and shows the expensive path did not establish the requested terminal condition. This repeated real task supports RQ1's multi-resource attribution hypothesis: one shared responsibility path reunites the SSH work across executions, while the selected additive measure changes its attributed importance. We do not claim that one resource measure universally dominates another.

Workload	Direct+ AgentProf	Direct+Raw +Evidence	Direct only	AgentProf only
AgentProcessBench	**.894**	.893	.863	.791
HINTBench	.517	**.518**	.411	.432
TraceElephant	**.326**	.324	.209	.259

Table 1: Standard per-query AP averaged as MAP over complete RQ2 populations. "Direct" is the benchmark-native trajectory judge/localizer. Both direct-first methods preserve every strict diagnostic ordering and retain identical source evidence. Higher is better.

RQ2: Problem Correspondence

RQ2 asks whether target-blind profiles place independently annotated problems in groups a developer can inspect early. We run complete AgentProcessBench, HINTBench, and TraceElephant workloads (Fan et al. 2026; Wang et al. 2026a; Chen et al. 2026a). Of HINTBench's reported 629 trajectories, the released test snapshot used here contains 536. A separate 80-trajectory validation snapshot selects one of 24 field orders before all 536 tests are scored.

Within each workload, we fix operations, target-blind paths, benchmark judge/localizer predictions, and scoring rules before loading test targets. AgentProcessBench averages judge votes within a group. HINTBench and TraceElephant assign each root-to-frame prefix the 95% Wilson lower bound (Wilson 1927) of its frozen positive-prediction fraction; an operation receives the maximum score over prefixes containing it.

The main test asks whether this profile adds information after a benchmark-native process judge or trajectory localizer has already scored the execution. We call its per-operation output the *direct diagnostic*: AgentProcessBench supplies released process-judge risk units, while HINTBench and TraceElephant supply trajectory-localizer decisions. In particular, the TraceElephant localizer reads the complete trace and reference answer before predicting the responsible agent and decisive step. The candidate ranks each operation lexicographically by (*direct diagnostic*, *Agent+Evidence group score*); hence the profile may refine exact diagnostic-score ties but cannot override a strict ordering made by the direct reader. One baseline uses the direct diagnostic alone. A stronger baseline replaces only the semantic-operation prefix with raw action while retaining the identical source-kind, tool/call, and outcome suffix and aggregation. AgentProf-only reports the profile component without the direct diagnostic.

Each query is one trajectory containing an annotated target. We report non-interpolated per-query AP and arithmetic-mean MAP across 614, 400, and 220 target-bearing queries, respectively (Robertson 2008). All 522 zero-positive trajectories are consumed for population coverage but excluded from MAP because AP is undefined without a relevant item.

Direct+AgentProf raises MAP over Direct-only by .031 [.024,.039], .107 [.093,.120], and .117 [.088,.148], using 10,000 paired resamples of trajectory clusters within benchmark-defined strata. Thus the profile adds clear rank-

ing information after a fixed trajectory reader on all three complete workloads (Table 1).

The information-matched Direct+Raw+Evidence refinement reaches .893, .518, and .324. Candidate-minus-baseline intervals are [-.0003,.0029], [-.0116,.0103], and [-.0247,.0280], so this experiment does not establish that the semantic-operation prefix ranks targets better when both views retain the same source evidence. The matched result attributes the ranking gain to group/evidence refinement in the complete profile, not specifically to its semantic prefix. Separately, that prefix supplies the cross-run hierarchy and source drilldown illustrated below. Because the local-first rule and fixed source-only paths were developed on these populations, this is adaptive mechanism evidence rather than untouched backend generalization.

Case Study 2: Differential Profiling at Scale

We next aggregate the complete mixed-outcome population from AgentRewardBench (Lù et al. 2025), rather than presenting one favorable trace pair. The collection contains 440 real trajectories over 125 tasks with both successful and unsuccessful runs, producing 338 complete bad–good pairs: 24 pair occurrences from AssistantBench, 102 from VisualWebArena, 144 from WebArena, and 68 from WorkArena. Because tasks can contain multiple runs, the 202 successful and 238 unsuccessful sessions are reused across pairs; the profile is therefore pair-occurrence weighted.

Before opening outcomes or expert annotations, three automatic Agent workers produce 2,131 sparse recursive annotations over all 7,229 source operations. The resulting hierarchy reaches semantic depth four before its LLM-call and tool-call evidence leaves. We then fold all 338 bad members and all 338 good members into one signed operation-count pprof, preserving the same source occurrences on both the recursive and fixed-chain projections.

The aggregate contains 7,366 bad-side and 3,780 good-side operation occurrences. Recovery accounts for 44.6% of bad-side occurrences but only 12.0% of good-side occurrences; completion accounts for 1.8% and 5.1%, respectively. The recursive recovery focus separates verification challenges, repeated searches, mistaken navigation, product or record retries, and repository inspection beneath the same responsibility, while source labels identify the contributing session and call.

We test whether this visible recovery structure corresponds to an independent problem rather than merely reflecting more steps. For each trajectory, recovery exposure is the fraction of source operations whose path contains the shared recovery responsibility. Among 435 trajectories with consensus expert looping labels, this score obtains AP .634 at prevalence .398; a 10,000-draw task-cluster bootstrap places AP minus prevalence in [.181,.293]. A registered fixed-chain repeated/error projection obtains AP .656, and the recursive-minus-fixed interval [-.107,.061] does not establish detector superiority. The recursive profile therefore contributes a source-drillable explanation of *where* recovery accumulates while independently corresponding to expert-identified looping.

This case answers RQ2 at collection scale: a target-blind recursive profile exposes failed-side recovery and successful-side completion under shared responsibilities, and its recovery signal corresponds to an independently annotated real problem. AgentRewardBench has no gold nested semantic hierarchy, so this result does not measure topology accuracy or causality.

Method	B^3 P	B^3 R	B^3 F1	Bound. F1
Automatic Agent (A2)	0.839	0.607	**0.704**	**0.394**
Multi-resolution recurrence	0.782	0.575	0.663	0.266
Native source tree	0.975	0.249	0.397	0.259
Source-native turn	0.983	0.221	0.361	0.246
Raw-action grouping	0.891	0.388	0.541	—

Table 2: Automatic operation-structure agreement on all 405 CodeTraceBench trajectories. B^3 scores the leaf partition; boundary F1 scores exact adjacent transitions.

RQ3: Automatic Operation Structure

RQ3 evaluates automatic backend outputs against independent annotations under each protocol's stated input boundary. Literal task and action fields use accuracy and macro-F1 (the unweighted mean of per-class F1) (Lewis et al. 2004). Partition-valued task, phase, and group fields use V-measure (Rosenberg and Hirschberg 2007) or ordinary B^3, and adjacent group boundaries use exact precision, recall, and F1 (Ruokolainen et al. 2016). Literal labels, permutation-invariant partitions, and adjacent boundaries are distinct outputs. A backend output is evaluated at the level it predicts: literal names, permutation-invariant leaf partitions, or adjacent interval boundaries. For legacy field-valued datasets, a mechanical adapter converts each predicted group into the same interval-annotation contract before AgentProf folds the original additive weights.

We first evaluate the default Agent-assisted backend on all 405 reconstructable CodeTraceBench trajectories (Li et al. 2026): 20,866 operations and 2,948 human-verified contiguous stages across four agent frameworks. Automatic Agents receive source-only packets and emit sparse complete-path marks over 17,148 source-native turns. A deterministic representation repair attaches a one-turn task-root-only prefix to its first nested responsibility, eliminating 149 artificial leaf groups without reading a stage label. The final 5,752 marks have semantic depth one (51 marks), two (5,608), or three (93); no prompt requests a depth. The current name replay then maps 5,537 open-vocabulary operation IDs to 1,434 action-first two- or three-word canonical IDs. It independently re-expands all operations from the unchanged marks, preserves the temporal occurrence and adjacent-boundary vectors exactly, and leaves no adjacent display-path collision. Nonadjacent and cross-session equal names intentionally merge so recurring work folds in pprof.

Ordinary operation-level B^3 measures the induced leaf partition, while exact adjacent-boundary F1 measures transition placement. Both are standard metrics and assign every operation equal weight. The recurrence and source-native controls receive the same operation sequence.

Automatic Agent annotation reaches 0.704 B^3 F1, improv-

Method	Prec.	Recall	Bound. F1	B^3 F1
Supervised predictor	0.700	0.782	**0.739**	**0.816**
Reference-calibrated recurrence	0.610	0.922	0.734	0.801
Label-free recurrence	0.592	0.799	0.680	0.786
Always boundary	0.476	1.000	0.645	0.678
Action change	0.386	0.626	0.477	0.659
Phase change	0.441	0.268	0.334	0.665

Table 3: RQ3 boundary and partition agreement on OSWorld-Human. Boundary F1 scores exact adjacent-pair decisions. B^3 scores predicted–human partitions.

ing over recurrence by 0.0414 (task-clustered bootstrap 95% interval [0.0214, 0.0606]) and over raw action by 0.163. Its boundary F1 is 0.394 versus 0.266 for recurrence. Thus the Agent-assisted backend is the strongest tested automatic constructor on this complete population, and its marks preserve exactly 20,866 operation counts and 494,862,929 provider-reported tokens when replayed at either width.

We test all 287 OSWorld-Human task-instance sessions (Abhyankar, Qi, and Zhang 2025) with complete group annotations. This population contains 3,978 operations, 3,691 adjacent pairs, and 2,042 groups. For the supervised Bernoulli Naive Bayes predictor (McCallum and Nigam 1998), we fix the model family, nine input fields, adjacent-pair features, and training procedure. Each session-held-out cross-validation fold fits its parameters and threshold on training sessions and predicts every held-out session once. Predicted boundaries become an ordinary group field that AgentProf projects and folds. With the same five folds, label-free recurrence uses only the other sessions' visible action transitions. OSWorld-Human supplies no non-redundant action detail, so the multi-resolution constructor exactly falls back to its coarse arm. Because we designed the recurrence rule after inspecting earlier results on this corpus, these results are development evidence rather than independent cross-family confirmation. Optional reference-calibrated recurrence uses training operations and group annotations to fit one NPMI cutoff, while held-out folds contribute only visible actions until predictions are fixed.

Controls place a boundary after every operation, visible action change, or visible phase change. We also report ordinary per-operation B^3 partition F1 (Bagga and Baldwin 1998), whose predicted–human group overlap captures merging and fragmentation.

The supervised predictor reaches 0.739 boundary F1 and 0.816 B^3 F1 versus 0.645 and 0.678 for the strongest controls. Label-free recurrence reaches 0.680 and 0.786, while reference-calibrated recurrence reaches 0.734 and 0.801 but requires group annotations that the default label-free method does not use. Every method conserves the total projected weight.

For target-blind TF-IDF/K-Means, V-measure (Rosenberg and Hirschberg 2007) over operation assignments reaches 0.557 on nine Mind2Web sessions (49 operations) and 0.815 on 100 ScienceWorld sessions (2,504 operations) (Deng et al. 2023; Wang et al. 2022). Every operation receives a prediction in both datasets, whereas a constant tag yields V-measure 0. Here, V-measure tests partitions, not literal names.

For task-family tags, we use a fixed evaluation-only Qwen3.6-27B llama.cpp backend distinct from the 3B CLI backend. Using only goals and declared family descriptions, it classifies all 1,012 AgentBoard goals into nine families (Qwen Team 2026; Ma et al. 2024). The backend reaches 0.695 macro-F1 and 0.733 accuracy, compared with 0.044 macro-F1 and 0.248 accuracy for the majority-class baseline, and produces identical assignments across three runs.

For action tags, a standalone backend-level adapter runs the fixed Qwen3.6-27B closed-label configuration using only the current thought/action and eight action definitions operationalized by TraceView (Qwen Team 2026; Sajadi et al. 2026). The adapter classifies all 2,737 published annotations from 120 AutoCodeRover, OpenHands, and RepairAgent trajectories (Bouzenia and Pradel 2025). The model reaches 0.498 macro-F1 and 0.628 accuracy, compared with 0.061 macro-F1 and 0.323 accuracy for the majority-class baseline. Its 0.437 macro-F1 gain has a whole-trajectory 95% bootstrap interval of [0.380, 0.494], and both complete runs agree exactly. Thirty-nine AutoCodeRover inputs contain the target word `Locate`. Excluding them leaves 0.490 macro-F1 and 0.622 accuracy, so the positive comparison to majority is unchanged.

Thus, across the named public populations, the automatic Agent constructor reaches 0.704 B^3 and 0.394 boundary F1 on CodeTraceBench; the label-free recurrence backend reaches 0.786 B^3 and 0.680 boundary F1 on OSWorld-Human; and closed-label task-family and action backends reach 0.695 and 0.498 macro-F1. These protocols evaluate complementary structural and literal outputs rather than collapsing them into one bespoke score.

RQ4: Profiling Cost

RQ4 separates fixed-input replay from the observable components of first profile construction. Across four complete public workloads and their union, we first measure the profiling core—parsing operation JSONL, constructing stacks, folding weights, and serializing the profile—by comparing the fixed semantic hierarchy `project, agent, task, phase, op, tool, status` with the raw hierarchy `project, agent, action, status` on identical inputs over three runs. Measurements use `agentpprof 0.2.37` on a 24-core Intel Core Ultra 9 285K with 125 GiB RAM and Linux 6.15.11.

Workload	Ops	Semantic (s)	Raw action (s)	Peak RSS (MiB)
AgentRewardBench	729	0.04	0.03	19.3
SATraj-OS	4,285	0.17	0.14	73.9
OSWorld-Human	6,010	0.24	0.21	109.1
AgentNet	16,741	0.70	0.58	279.3
Union	27,765	1.16	0.97	465.2

Table 4: RQ4 profile-construction cost. Times are three-run medians, and resident set size (RSS) is the largest peak observed for the semantic profile.

Both time curves are monotonic across the five input sizes.

The semantic curve has a descriptive slope of 0.0418 ms per operation ($R^2 = 0.9997$), reaching 23,935 operations/s on the union. For the 27,765-operation union, semantic construction completes in 1.16 seconds, reaches 465.2 MiB peak RSS, and incurs 190 ms (19.6%) more time and 5.25 MiB (1.14%) more peak RSS than raw-action grouping. We then reconstruct the adopted A2 path three times on all 405 CodeTrace sessions (17,148 source-native turns and 20,866 operations). Joining the fixed target operations to released raw archives and constructing source-only Agent packets takes 501.64 s median. Deterministic annotation assembly, root-prefix repair, validation, and name canonicalization take 3.54 s. Afterward, the current source-preserving stack constructs either the operation-count or 494,862,929-token profile in 1.17 s median. Every reconstructed population and A2 artifact is byte-identical across runs; all profiles load in stock pprof and conserve exact mass.

The existing automatic-Agent annotations were produced in two disjoint workflow waves whose packet-manifest-to-final-artifact spans sum to 54.36 minutes. We report this only as an artifact-time workflow envelope: retained telemetry cannot separate model inference from orchestration, idle time, and file writing or recover provider token usage. Thus 1.17 s is replay latency, not annotation latency; the measured deterministic first-construction components total 506.35 s, while model/provider inference remains outside the instrumented timing.

Scope and Limitations

Results apply to the named populations and annotation protocols. CodeTraceBench is the complete development population for the Agent-assisted constructor; OSWorld-Human separately evaluates recurrence and supervised boundary backends. RQ2 scores released localizer predictions over complete public workloads; it does not measure end-user diagnosis time. RQ4 measures fixed-input replay and the deterministic A2 path from normalized operations plus released raw archives. It excludes capture, raw-to-normalized conversion, live-agent overhead, and instrumented model/provider annotation timing; the reported 54.36-minute artifact envelope is not model latency.

Related Work

Agent observability. Platforms aggregate spans, metadata, and evaluations (LangChain 2024; Langfuse 2024; OpenTelemetry 2024; Datadog 2025). Datadog Patterns and LangSmith Insights derive cross-trace hierarchies and roll up metrics, NeMo profiles instrumented workflows, and OpenTelemetry Profiles links profiles to traces (Datadog 2026; LangChain 2026; NVIDIA 2026; OpenTelemetry 2026). These systems establish grouping, rollups, profile naming and compatibility, and trace linkage. AgentProf complements these systems by recursively assigning shared semantic responsibility over a preserved source-call tree and folding conserved resource measures into standard pprof profiles.

Cross-run agent analysis. TraceProbe normalizes coding traces into canonical actions and deterministic process diagnostics, while Graphectory builds per-trajectory cyclic action/navigation graphs and aggregates phase patterns (Shu et al. 2026; Liu et al. 2026a). Act·onomy supplies a fixed three-level behavior taxonomy and automated behavioral profiles; CHIEF instead builds task/subtask causal graphs for oracle-guided failure attribution (Gao et al. 2026; Wang et al. 2026b). Hodoscope compares behavior distributions to direct human review, while TraceGraph builds shared decision landscapes that guide recovery (Zhong, Saxena, and Raghunathan 2026; Nian et al. 2026). Together, these systems establish canonical actions, recurring cross-run structure, and analysis-to-decision precedents. AgentProf complements them with shared variable-depth responsibility annotations that cover native call evidence, fold across runs, and replay the same boundaries under different additive resource measures in a standard profiler.

Profiling and diagnosis. pprof and flame graphs fold runtime stacks, whereas Pivot Tracing dynamically selects and groups measurements across causally related events (Google 2024; Gregg 2011; Mace, Roelke, and Fonseca 2015). AgentRx diagnoses failures, CodeTracer reconstructs per-run states, and localization benchmarks score faulty steps, agents, or expert-supported perspectives (Barke et al. 2026; Li et al. 2026; Fan et al. 2026; Wang et al. 2026a; Chen et al. 2026a; In et al. 2026). AgentProf applies the profiler's population-level principle to agents by annotating recurring semantic responsibility over completed trajectories and folding it without requiring code identity or treating runtime nesting as semantic truth.

Conclusion

Agent observability needs profiling, not only debugging. AgentProf recursively annotates semantic responsibility over source-native traces and folds covered source evidence under shared operation stacks across runs. Its profiles expose resource-dependent bottlenecks and success–failure differences, improve stage-partition agreement on CodeTraceBench, supplement benchmark-native direct diagnostics on all three localization workloads, and recover OSWorld-Human groups at 0.680 boundary F1 and 0.786 B^3 F1. After marks are fixed, AgentProf builds a 27,765-operation profile in 1.16 seconds, making population profiling practical alongside per-run debugging.

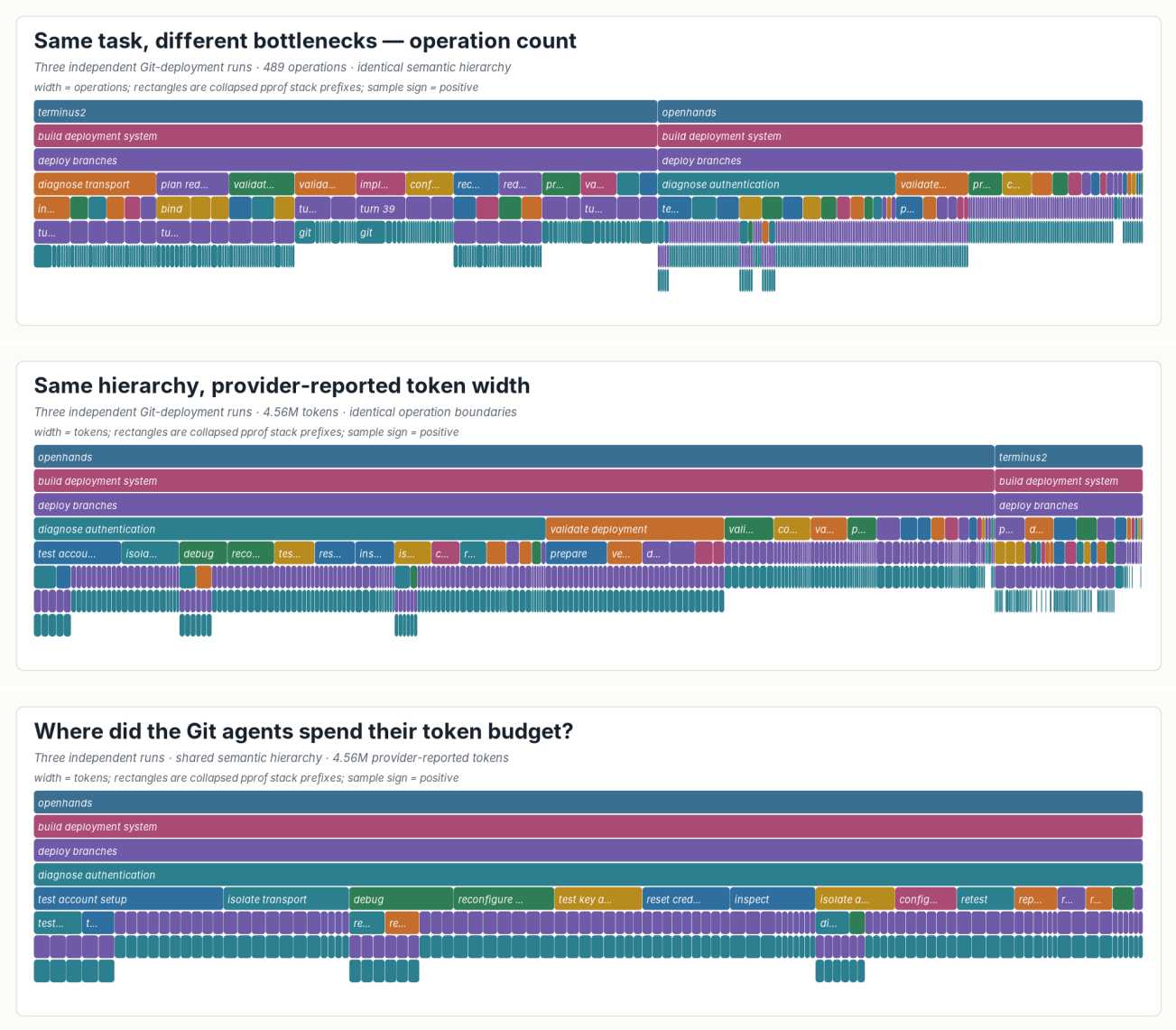

Figure 2: All three real `git-multibranch` executions under one fixed operation hierarchy. Width is operation count (top) or provider-reported tokens (middle); the bottom panel focuses the shared `diagnose authentication` operation and exposes its recursively nested hypotheses, controls, LLM calls, and tool leaves. AgentPProf emits only standard pprof; a paper-only renderer reads the same `.pb.gz` through `go tool pprof` without introducing a product frontend.

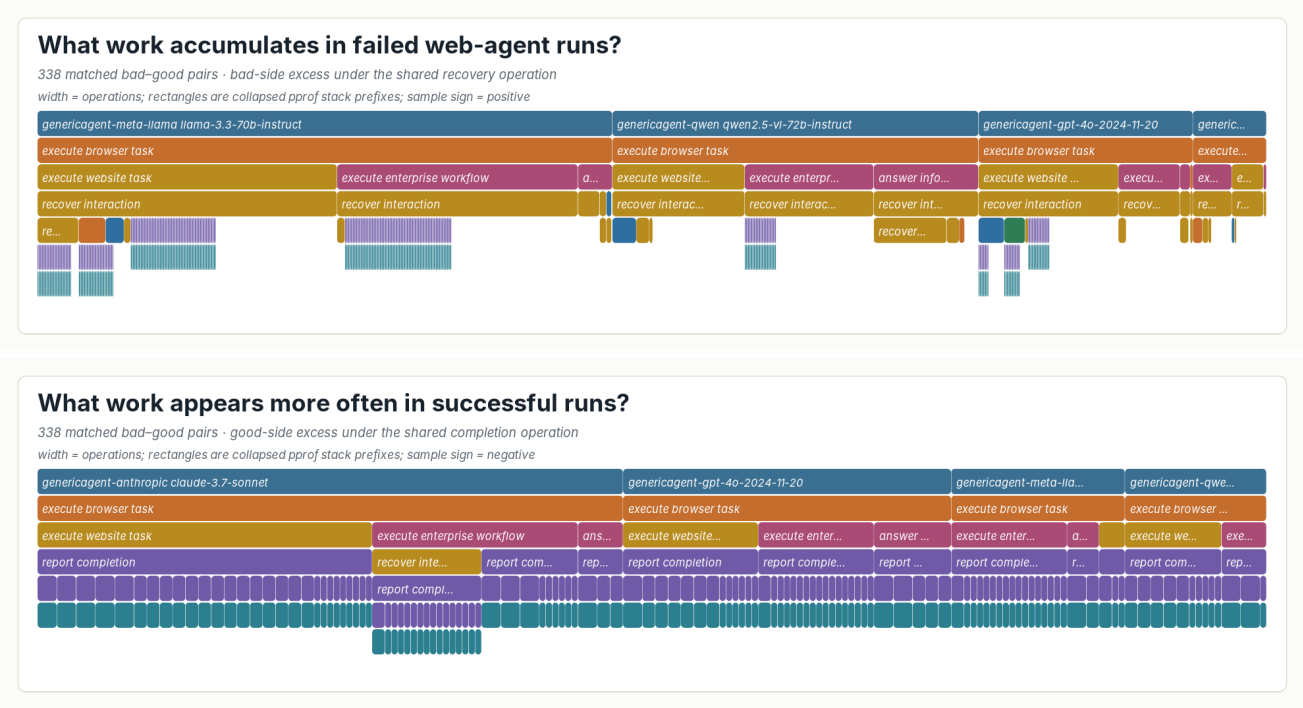

Figure 3: Focused paper renderings of the signed pprof profile over all 338 bad–good pairs. The top panel selects positive samples under `recover interaction`: 3,286 bad-side versus 455 good-side occurrences. The bottom selects negative samples under `report completion`: 135 bad-side versus 191 good-side occurrences. Each path descends from shared responsibilities to contributing LLM and tool calls. These are aggregate diagnostic differences, not causal effects.

References

Abhyankar, R.; Qi, Q.; and Zhang, Y. 2025. OSWorld-Human: Benchmarking the Efficiency of Computer-Use Agents. arXiv preprint arXiv:2506.16042.

Anthropic. 2025. Claude Code: An Agentic Coding Tool. https://code.claude.com/docs/en/overview.

Arize AI. 2024. OpenInference Specification. https://arize-ai.github.io/openinference/spec/.

Bagga, A.; and Baldwin, B. 1998. Entity-Based Cross-Document Coreferencing Using the Vector Space Model. In *36th Annual Meeting of the Association for Computational Linguistics and 17th International Conference on Computational Linguistics, Volume 1*, 79–85. Montreal, Quebec, Canada: Association for Computational Linguistics.

Barke, S.; Goyal, A.; Khare, A.; Singh, A.; Nath, S.; and Bansal, C. 2026. AgentRx: Diagnosing AI Agent Failures from Execution Trajectories. arXiv preprint arXiv:2602.02475.

Bouma, G. 2009. Normalized (Pointwise) Mutual Information in Collocation Extraction. In *From Form to Meaning: Processing Texts Automatically, Proceedings of the Biennial GSCL Conference 2009*, 31–40. Tübingen, Germany.

Bouzenia, I.; and Pradel, M. 2025. Understanding Software Engineering Agents: A Study of Thought-Action-Result Trajectories. In *2025 IEEE/ACM 40th International Conference on Automated Software Engineering (ASE)*, 2846–2857. IEEE.

Chen, M.; Wang, J.; Mu, F.; Wang, Y.; Liu, Z.; Feng, H.; and Wang, Q. 2026a. Seeing the Whole Elephant: A Benchmark for Failure Attribution in LLM-Based Multi-Agent Systems. arXiv preprint arXiv:2604.22708.

Chen, X.; Yin, Z.; He, S.; Huang, B.; Lei, S.; et al. 2026b. Safactory: A Scalable Agentic Infrastructure for Training Trustworthy Autonomous Intelligence. arXiv preprint arXiv:2605.06230.

Datadog. 2025. LLM Observability. https://docs.datadoghq.com/llm_observability/.

Datadog. 2026. LLM Observability Patterns. https://docs.datadoghq.com/llm_observability/monitoring/patterns/.

Deng, X.; Gu, Y.; Zheng, B.; Chen, S.; Stevens, S.; Wang, B.; Sun, H.; and Su, Y. 2023. Mind2Web: Towards a Generalist Agent for the Web. In *Advances in Neural Information Processing Systems 36 (NeurIPS), Datasets and Benchmarks Track*.

Fan, S.; Ye, X.; Huo, Y.; Chen, Z.-Y.; Guo, Y.; Yang, S.; Yang, W.; Ye, S.; Chen, J.; Chen, H.; Cong, X.; and Lin, Y. 2026. AgentProcessBench: Diagnosing Step-Level Process Quality in Tool-Using Agents. In *Proceedings of KDD*.

Gao, J.; Sun, K.; Huang, J.-t.; Van Koevering, K.; Ji, S.; Huang, H.; Shi, W.; Lu, Z.; Xiao, Z.; Khashabi, D.; and Dredze, M. 2026. How to Interpret Agent Behavior. arXiv:2605.13625.

Google. 2024. pprof. https://github.com/google/pprof.

Gregg, B. 2011. Flame Graphs. https://www.brendangregg.com/flamegraphs.html.

In, Y.; Tanjim, M.; Subramanian, J.; Kim, S.; Bhattacharya, U.; Kim, W.; Park, S.; Sarkhel, S.; and Park, C. 2026. Rethinking Failure Attribution in Multi-Agent Systems: A Multi-Perspective Benchmark and Evaluation. arXiv:2603.25001.

LangChain. 2024. LangSmith Observability. https://docs.langchain.com/langsmith/observability.

LangChain. 2026. LangSmith Insights. https://docs.langchain.com/langsmith/insights.

Langfuse. 2024. Langfuse Observability. https://langfuse.com/docs/observability/overview.

Lewis, D. D.; Yang, Y.; Rose, T. G.; and Li, F. 2004. RCV1: A New Benchmark Collection for Text Categorization Research. *Journal of Machine Learning Research*, 5: 361–397.

Li, H.; Yao, Y.; Zhu, L.; Feng, R.; Ye, H.; Wang, J.; He, Y.; Zou, P.; Zhang, L.; Lei, X.; Huang, H.; Deng, K.; Sun, M.; Zhang, Z.; Ye, H.; and Liu, J. 2026. CodeTracer: Towards Traceable Agent States. arXiv:2604.11641.

Li, M.; Zhao, Y.; Yu, B.; Song, F.; Li, H.; Yu, H.; Li, Z.; Huang, F.; and Li, Y. 2023. API-Bank: A Comprehensive Benchmark for Tool-Augmented LLMs. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 3102–3116.

Li, W.; Bishop, W. E.; Li, A.; Rawles, C.; Campbell-Ajala, F.; Tyamagundlu, D.; and Riva, O. 2024. On the Effects of Data Scale on UI Control Agents. In *NeurIPS 2024, Datasets and Benchmarks Track*.

Liu, S.; Chen, Y.; Krishna, R.; Sinha, S.; Ganhotra, J.; and Jabbarvand, R. 2026a. Process-Centric Analysis of Agentic Software Systems. *Proceedings of the ACM on Programming Languages*, 10(OOPSLA1): 1961–1988.

Liu, Z.; Xie, J.; Ding, Z.; Li, Z.; Yang, B.; et al. 2026b. ScaleCUA: Scaling Open-Source Computer Use Agents with Cross-Platform Data. In *Proceedings of ICLR*.

Lu, Q.; Shao, W.; Liu, Z.; Meng, F.; Li, B.; Chen, B.; Huang, S.; Zhang, K.; Qiao, Y.; and Luo, P. 2025. GUIOdyssey: A Comprehensive Dataset for Cross-App GUI Navigation on Mobile Devices. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*.

Lù, X. H.; Kasner, Z.; and Reddy, S. 2024. WebLINX: Real-World Website Navigation with Multi-Turn Dialogue. In *Proceedings of ICML*, volume 235 of *Proceedings of Machine Learning Research*, 33007–33056. PMLR.

Lù, X. H.; Kazemnejad, A.; Meade, N.; Patel, A.; Shin, D.; Zambrano, A.; Stańczak, K.; Shaw, P.; Pal, C. J.; and Reddy, S. 2025. AgentRewardBench: Evaluating Automatic Evaluations of Web Agent Trajectories. In *Conference on Language Modeling (COLM)*.

Ma, C.; Zhang, J.; Zhu, Z.; Yang, C.; Yang, Y.; Jin, Y.; Lan, Z.; Kong, L.; and He, J. 2024. AgentBoard: An Analytical Evaluation Board of Multi-turn LLM Agents. In *Advances in Neural Information Processing Systems*, volume 37, 74325–74362. Curran Associates, Inc.

Mace, J.; Roelke, R.; and Fonseca, R. 2015. Pivot Tracing: Dynamic Causal Monitoring for Distributed Systems. In *Proceedings of the 25th Symposium on Operating Systems Principles*, 378–393.

MacQueen, J. B. 1967. Some Methods for Classification and Analysis of Multivariate Observations. In *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability*, volume 1, 281–297. University of California Press.

McCallum, A.; and Nigam, K. 1998. A Comparison of Event Models for Naive Bayes Text Classification. In *AAAI-98 Workshop on Learning for Text Categorization*, 41–48. AAAI Press.

Nian, J.; Chen, K.; Zhang, G.; Cao, Y.; and Jiang, Y. 2026. TraceGraph: Shared Decision Landscapes for Diagnosing and Improving Agent Trajectories. arXiv:2605.31308.

NVIDIA. 2026. Profiling and Performance Monitoring of NVIDIA NeMo Agent Toolkit Workflows. Official documentation.

OpenAI. 2025. OpenAI Codex CLI. GitHub repository.

OpenTelemetry. 2024. OpenTelemetry GenAI Semantic Conventions. https://github.com/open-telemetry/semantic-conventions-genai.

OpenTelemetry. 2026. OpenTelemetry Profiles. Official specification.

Qin, Y.; Liang, S.; Ye, Y.; Zhu, K.; Yan, L.; Lu, Y.; Lin, Y.; Cong, X.; Tang, X.; Qian, B.; Zhao, S.; Hong, L.; Tian, R.; Xie, R.; Zhou, J.; Gerber, M.; Li, D.; Liu, Z.; and Sun, M. 2024. ToolLLM: Facilitating Large Language Models to Master 16000+ Real-world APIs. In *Proceedings of the 12th International Conference on Learning Representations (ICLR)*.

Qwen Team. 2026. Qwen3.6-27B: Flagship-Level Coding in a 27B Dense Model. Official model card.

Robertson, S. 2008. A New Interpretation of Average Precision. In *Proceedings of the 31st Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*, 689–690. ACM.

Rosenberg, A.; and Hirschberg, J. 2007. V-Measure: A Conditional Entropy-Based External Cluster Evaluation Measure. In *Proceedings of the 2007 Joint Conference on Empirical Methods in Natural Language Processing and Computational Natural Language Learning*, 410–420. Prague, Czech Republic: Association for Computational Linguistics.

Ruokolainen, T.; Kohonen, O.; Sirts, K.; Grönroos, S.-A.; Kurimo, M.; and Virpioja, S. 2016. A Comparative Study of Minimally Supervised Morphological Segmentation. *Computational Linguistics*, 42(1): 91–120.

Sajadi, A.; Nguyen, T.; Huynh, K.; Parra, E.; and Chatterjee, P. 2026. TraceView: Interactive Visualization of Agentic Program Repair Trajectories. arXiv:2606.22110.

Shu, R.; Chong, C. Y.; Zhou, X.; Peng, Y.; Wu, Z.; Han, X.; Zhuang, Z.; Yuan, G.; and Wang, Y. 2026. What Resolve Rate Hides: Trajectory Structure Diagnostics for Coding Agents. arXiv:2607.06184.

Wang, J.; Hou, J.; Wang, F.; Jian, P.; Bao, C.; and Lv, Z. 2026a. HINTBench: Horizon-Agent Intrinsic Non-Attack Trajectory Benchmark. arXiv preprint arXiv:2604.13954.

Wang, R.; Jansen, P.; Côté, M.-A.; and Ammanabrolu, P. 2022. ScienceWorld: Is Your Agent Smarter than a 5th Grader? In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, 11279–11298. Association for Computational Linguistics.

Wang, X.; Wang, B.; Lu, D.; Yang, J.; Xie, T.; et al. 2025. AgentNet. https://huggingface.co/datasets/xlangai/AgentNet.

Wang, Y.; Wu, W.; Wang, J.; and Wang, Q. 2026b. From Flat Logs to Causal Graphs: Hierarchical Failure Attribution for LLM-based Multi-Agent Systems. arXiv:2602.23701.

Wilson, E. B. 1927. Probable Inference, the Law of Succession, and Statistical Inference. *Journal of the American Statistical Association*, 22(158): 209–212.

Xie, T.; Zhang, D.; Chen, J.; Li, X.; Zhao, S.; Cao, R.; Hua, T. J.; Cheng, Z.; Shin, D.; Lei, F.; Liu, Y.; Xu, Y.; Zhou, S.; Savarese, S.; Xiong, C.; Zhong, V.; and Yu, T. 2024. OSWorld: Benchmarking Multimodal Agents for Open-Ended Tasks in Real Computer Environments. In *Advances in Neural Information Processing Systems 37 (NeurIPS), Datasets and Benchmarks Track*.

Xu, Y.; Lu, D.; Shen, Z.; Wang, J.; Wang, Z.; Mao, Y.; Xiong, C.; and Yu, T. 2025. AgentTrek: Agent Trajectory Synthesis via Guiding Replay with Web Tutorials. In *Proceedings of ICLR*.

Yang, J.; Jimenez, C. E.; Wettig, A.; Lieret, K.; Yao, S.; Narasimhan, K.; and Press, O. 2024. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. In *Advances in Neural Information Processing Systems 37 (NeurIPS)*.

Yao, S.; Chen, H.; Yang, J.; and Narasimhan, K. 2022. WebShop: Towards Scalable Real-World Web Interaction with Grounded Language Agents. In *Advances in Neural Information Processing Systems 35 (NeurIPS)*.

Yao, S.; Shinn, N.; Razavi, P.; and Narasimhan, K. 2025. τ-bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains. In *The Thirteenth International Conference on Learning Representations (ICLR)*.

Zheng, L.; Chiang, W.-L.; Sheng, Y.; Zhuang, S.; Wu, Z.; Zhuang, Y.; Lin, Z.; Li, Z.; Li, D.; Xing, E. P.; Zhang, H.; Gonzalez, J. E.; and Stoica, I. 2023. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems 36 (NeurIPS), Datasets and Benchmarks Track*.

Zheng, Y.; Hu, Y.; Yu, T.; and Quinn, A. 2025. AgentSight: System-Level Observability for AI Agents Using eBPF. In *Proceedings of the 4th Workshop on Practical Adoption Challenges of ML for Systems*.

Zhong, Z.; Saxena, S.; and Raghunathan, A. 2026. Hodoscope: Unsupervised Monitoring for AI Misbehaviors. arXiv:2604.11072.